



Programação web

Aprimorando nossa API

Prof. Diego Cardoso Borda Castro



Resumo

- Autenticação
- Autorização
- Camadas Nest
- OpenAPI
- Task scheduling
- CQRS
- Eventos
- Queues
- Caching
- Threads



Autenticação

- O que é Autenticação?
 - Restringir acesso
 - Rotas privadas
 - Desanonimização



Authorization

What you can do



Authentication

Who you are

- Métodos
 - Senha
 - Token
 - JWT
 - OAuth2
 - Multifator (MFA)
 - Single Sign-On



Autenticação

- Fluxo
 - Login
 - Envio de credenciais para o servidor
 - Geração do token
 - Sessão
 - Armazenar dados ou token
 - LocalStorage
- Refresh
 - Aumentar a segurança
 - Atualizar token
- Logout
 - Remover tokens e sessões

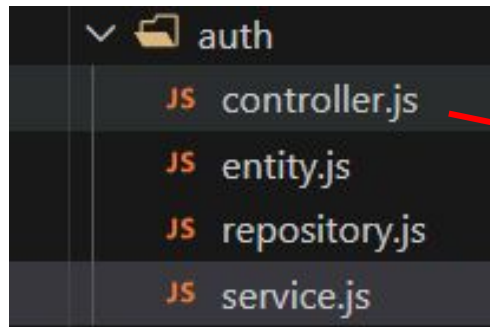


Autenticação

- Implementação
 - Rotas protegidas
 - Middleware ou guards para validar o token
 - Criar usuário
 - Login
- npm i bcrypt
 - Hash
- npm i jsonwebtoken
 - Geração do token



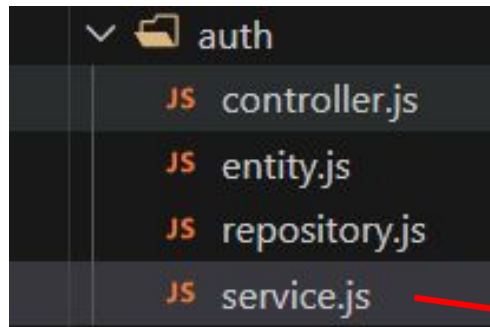
Autenticação



```
1 import * as authService from "../service.js";
2
3 export async function criarUsuario(req, res) {
4   const { username, password } = req.body;
5
6   try {
7     const aluno = await authService.criarUsuario(username, password);
8
9     res.status(201).json({ message: 'Usuário criado!' });
10  } catch (error) {
11    if(process.env.NODE_ENV == "dev")
12      console.log(error);
13    res.status(500).json({ message: 'Erro ao criar o usuário' });
14  }
15 }
16
```



Autenticação



```
1 import * as authRepo from "../repository.js";
2 import * as bcrypt from "bcrypt"
3
4 export async function criarUsuario(username, password) {
5     const salto = await bcrypt.genSalt(10);
6     const hash = await bcrypt.hash(password, salto);
7
8     const novoUsuario = { username, password: hash };
9
10    return authRepo.create(novoUsuario);
11 }
```



Autenticação

Data Output	Messages	Notifications
id [PK] bigint	username character varying (255)	password character varying (255)
1	Diego	\$2b\$10\$6LZtRRkoHuaR38NBV6hq9Or0shdzoOPYTso/cl0511oto2uA0IH...

```
1 import { Usuario } from "../entity.js";
2
3 export async function create(usuario) {
4   return Usuario.create({
5     username: usuario.username,
6     password: usuario.password
7   })
8 }
```

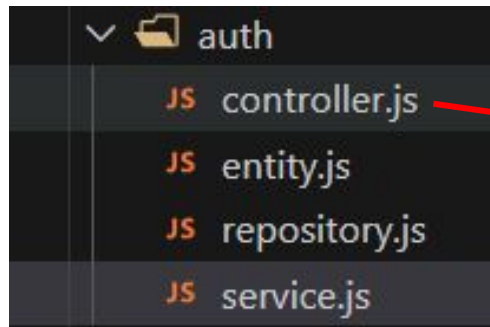
Repositório

```
1 import { Model, DataTypes } from "sequelize";
2 import { sequelize } from "../sequelize.js";
3
4 export class Usuario extends Model {}
5
6 Usuario.init(
7   {
8     id: {
9       type: DataTypes.BIGINT,
10      primaryKey: true,
11      autoIncrement: true
12    },
13    username: {
14      type: DataTypes.STRING
15    },
16    password: {
17      type: DataTypes.STRING
18    }
19  },
20  {
21    sequelize,
22    modelName: "Usuario",
23    tableName: "usuario",
24    timestamps: false
25  }
26 )
```

Service



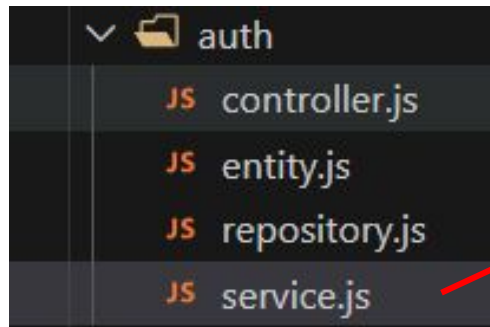
Autenticação



```
1  export async function gera_token(req, res) {
2    try {
3      const { username, password } = req.body;
4      const token = await authService.gera_token(username, password);
5
6      if(!token)
7        res.status(400).json({ message: 'Senha ou usuário incorreto!' });
8
9      res.status(200).json({ token });
10   } catch (error) {
11     if(process.env.NODE_ENV == "dev")
12       console.log(error);
13     res.status(500).json({ message: 'Erro ao realizar o login!' });
14   }
15 }
```



Autenticação



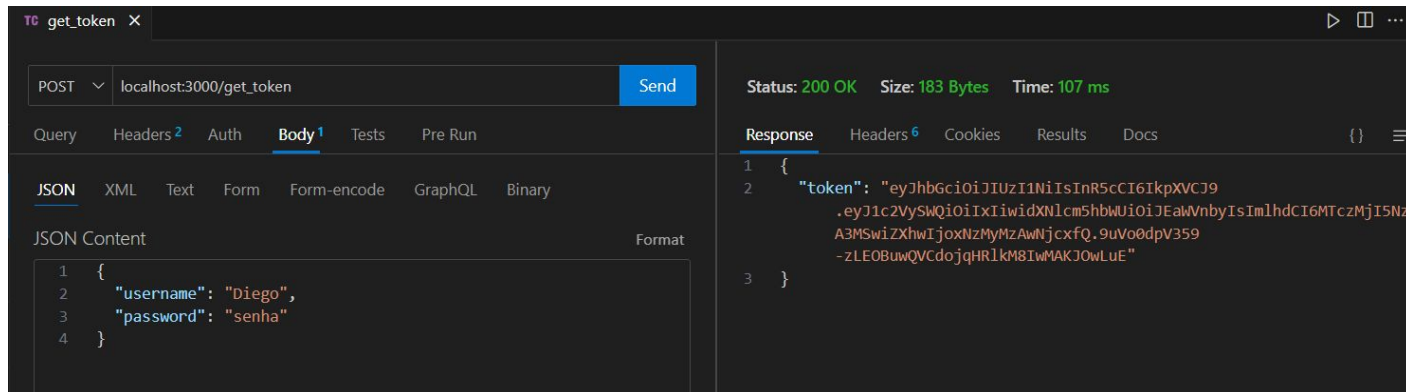
```
1 export async function gera_token(username, password) {
2   const usuario = await authRepo.findByUsername(username);
3
4   const validacao = await bcrypt.compare(password, usuario.password);
5
6   if(!validacao)
7     return null;
8
9   const token = jwt.sign(
10     { userId: usuario.id, username: usuario.username },
11     process.env.JWT_SECRET,
12     { expiresIn: '1h' }
13   );
14
15   return token;
16 }
```



Autenticação



```
1 export function findByUsername(username) {  
2  
3   return Usuario.findOne({  
4     where: {  
5       username: username  
6     }  
7   });  
8 }
```





Middleware de Autenticação

```
1 import authMiddleware from './middlewares/auth.js';
2
3 const app = express();
4 const port = 3000;
5
6 app.use(express.json());
7
8 app.post("/aluno", authMiddleware, criarAluno);
9 app.get("/alunos", authMiddleware, listarAlunos);
10 app.get("/aluno/:id", visualizarAluno);
11 app.put("/aluno/:id", atualizarAluno);
12 app.delete("/aluno/:id", deletarAluno);
13
14 app.post("/usuario", criarUsuario);
15 app.post("/get_token", gera_token);
```

```
1 import jwt from "jsonwebtoken"
2
3 const authMiddleware = (req, res, next) => {
4   const token = req.headers['authorization'];
5
6
7   if (!token) {
8     return res.status(403).json({ message: 'Acesso negado!' });
9   }
10
11   try {
12     const decoded = jwt.verify(
13       token,
14       process.env.JWT_SECRET);
15     req.user = decoded;
16     next();
17   } catch (error) {
18     res.status(401).json({ message: 'Acesso Negado!' });
19   }
20 };
21
22 export default authMiddleware;
```



Middleware de Autenticação

TC listar alunos X

GET localhost:3000/alunos Send

Query Headers 2 Auth Body Tests Pre Run

HTTP Headers ☐ Raw

☒	Accept	*/*
☒	User-Agent	Thunder Client (https://www.thunderclient.ı
☐	authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1
☐	header	value

Status: 403 Forbidden Size: 28 Bytes Time: 6 ms

Response Headers 6 Cookies Results Docs

```
1 {
2   "message": "Acesso negado!"
3 }
```



Middleware de Autenticação

TC listar alunos X

GET localhost:3000/alunos Send

Query Headers 3 Auth Body Tests Pre Run

HTTP Headers ☐ Raw

☒	Accept	*/*
☒	User-Agent	Thunder Client (https://www.thunderclient....
☒	authorization	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1
☐	header	value

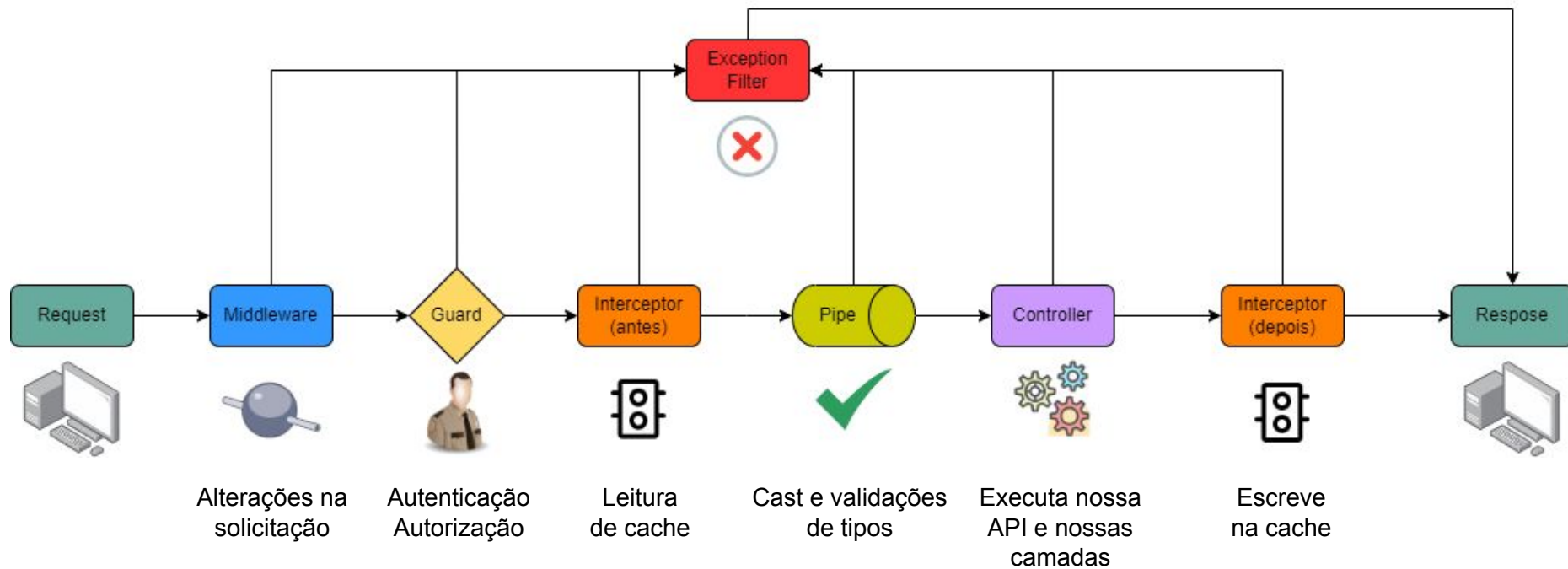
Status: 200 OK Size: 87 Bytes Time: 66 ms

Response Headers 6 Cookies Results Docs

```
1 [
2   {
3     "id": 2,
4     "nome": "Cardoso",
5     "matricula": "456"
6   },
7   {
8     "id": 1,
9     "nome": "Diego",
10    "matricula": "123"
11  }
12 ]
```



Novas camadas (Nest)





OpenAPI / Swagger

- `npm install --save @nestjs/swagger`

```
const config = new DocumentBuilder()
  .setTitle('Cats example')
  .setDescription('The cats API description')
  .setVersion('1.0')
  .addTag('cats')
  .build();
const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('api', app, document);
```




OpenAPI / Swagger

```
@Post()  
create(@Body() createAlunoDto: CreateAlunoDto) {  
  return this.alunoService.create(createAlunoDto);  
}
```

Schemas

```
CreateAlunoDto ▾ {  
  nome*           string  
                  minimum: 10  
                  maximum: 50  
                  Nome do aluno  
  matricula*      string  
}
```

```
import { ApiProperty } from "@nestjs/swagger";  
  
export class CreateAlunoDto {  
  @ApiProperty({  
    description: 'Nome do aluno',  
    minimum: 10,  
    maximum: 50  
  })  
  nome: string;  
  
  @ApiProperty()  
  matricula: string;  
}
```



OpenAPI / Swagger

- npm install class-validator class-transformer

```
@Get()
findAll(@Query() filter: FilterAlunoDto) {
  return this.alunoService.findAll();
}
```

GET /aluno

Parameters

Name	Description
nome string (query)	Nome Example : Diego
matricula string (query)	Mtrícula Example : 123456

Diego

123456

```
export class FilterAlunoDto {
  @ApiPropertyOptional({
    description: 'Nome',
    example: 'Diego'
  })
  @IsOptional()
  @IsString()
  nome?: string;

  @ApiPropertyOptional({
    description: 'Mtrícula',
    example: '123456'
  })
  @IsOptional()
  @IsString()
  matricula?: string;
}
```



OpenAPI / Swagger

- @ApiTags('Alunos')

Alunos		^
POST	/aluno	v
GET	/aluno	v

- Segurança
 - @ApiBasicAuth()
 - @ApiSecurity('basic')
 - @ApiBearerAuth()



Task scheduling

- cron
- `npm install --save @nestjs/schedule`

```
import { Module } from '@nestjs/common';
import { ScheduleModule } from '@nestjs/schedule';

@Module({
  imports: [
    ScheduleModule.forRoot()
  ],
})
export class AppModule {}
```

```
* * * * *
| | | | |
| | | | | dia da semana
| | | | | meses
| | | dia do mês
| | horas
| minutos
segundos (opcional)
```



Task scheduling

```
//"* * * * *"  
@Cron(CronExpression.EVERY_SECOND)  
@Interval(1000)  
@Timeout(5000)
```

```
@Module({  
  controllers: [AlunoController],  
  providers: [AlunoService, AlunoScheduler],  
})  
export class AlunoModule {}
```

```
@Injectable()  
export class AlunoScheduler {  
  private readonly logger = new Logger(AlunoScheduler.name);  
  
  //"* * * * *"  
  @Cron(CronExpression.EVERY_SECOND)  
  verificar() {  
    //lógica para buscar e cancelar os pedidos.  
    this.logger.debug(  
      `Aluno ${Math.floor(Math.random() * 100)} verificado!`,  
    );  
  }  
}
```

EMS	OUTPUT	DEBUG CONSOLE	TERMINAL	PORTS
t] 2840	- 16/05/2024, 21:33:26	DEBUG [AlunoScheduler]	Aluno 23 verificado!	
t] 2840	- 16/05/2024, 21:33:27	DEBUG [AlunoScheduler]	Aluno 49 verificado!	
t] 2840	- 16/05/2024, 21:33:28	DEBUG [AlunoScheduler]	Aluno 36 verificado!	
t] 2840	- 16/05/2024, 21:33:29	DEBUG [AlunoScheduler]	Aluno 40 verificado!	
t] 2840	- 16/05/2024, 21:33:30	DEBUG [AlunoScheduler]	Aluno 57 verificado!	
t] 2840	- 16/05/2024, 21:33:31	DEBUG [AlunoScheduler]	Aluno 13 verificado!	



CQRS

- CQS
 - Command and Query Separation
 - FindOne(id)
 - FindByName(nome)
 - FindAll()
 - Insert(dado)
 - Update(id, dado)
 - Delete(id)



CQRS

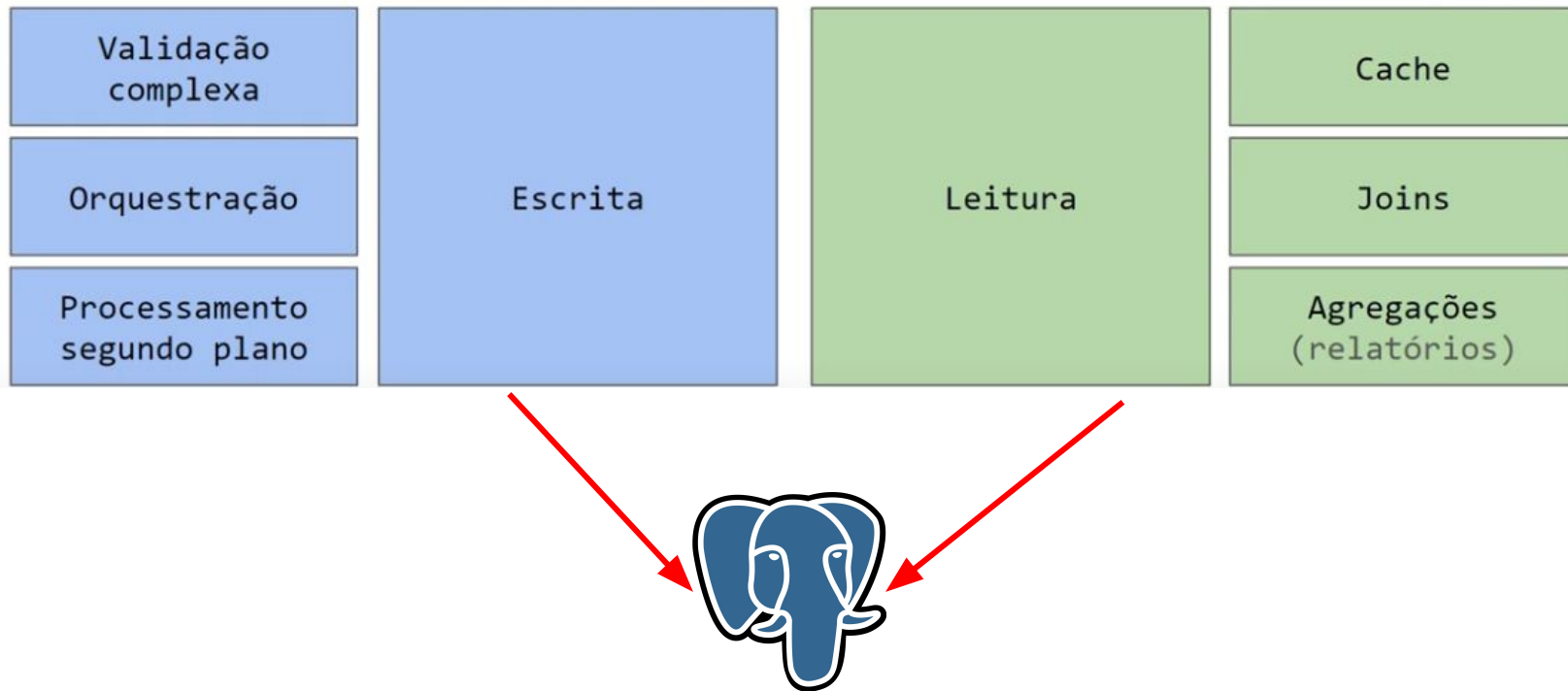
- CQRS
 - Command and Query Responsibility Segregation
 - Pega o CQS e leva pro nível da aplicação
 - Repositório
 - Lendo e escrevendo

Leitura

Escrita

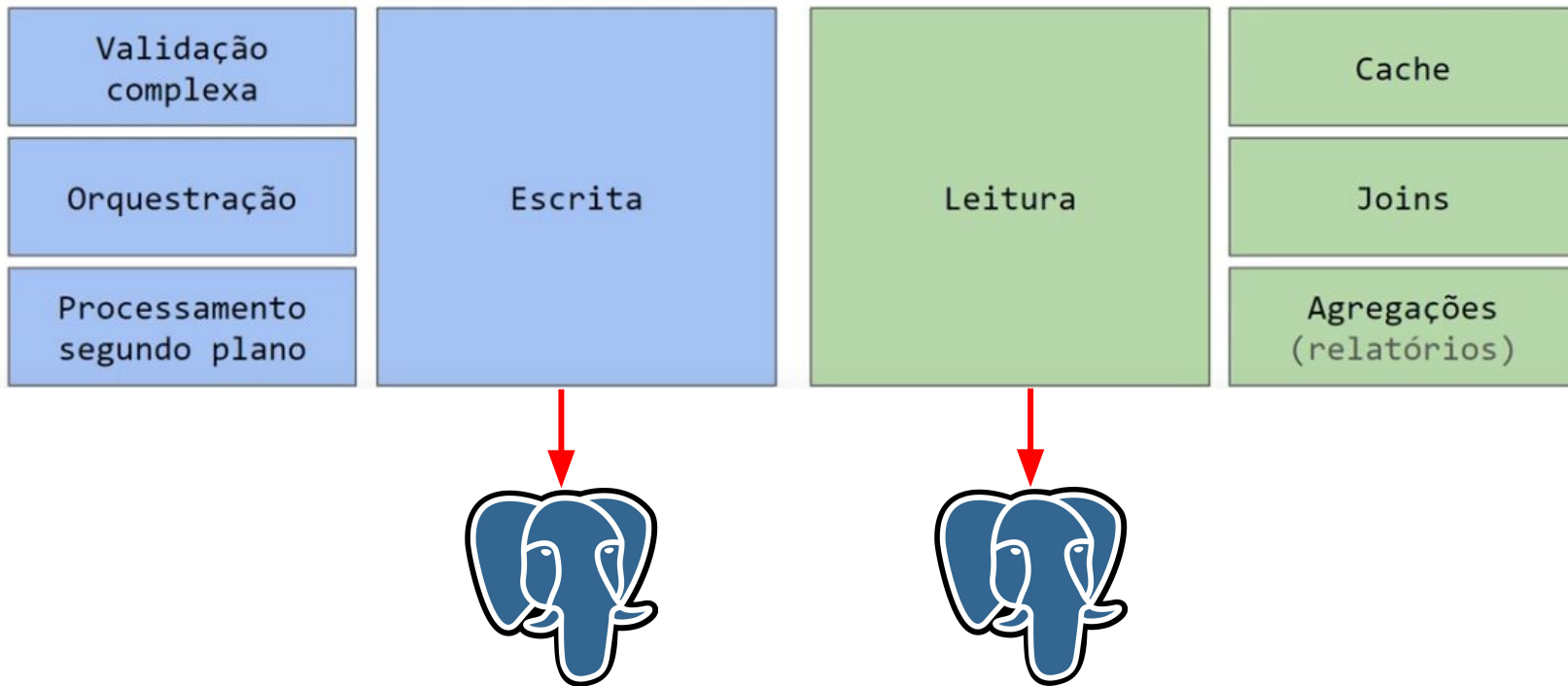


CQRS





CQRS





CQRS

Leitura

- FindOne(id)
- FindByName(nome)
- FindAll()

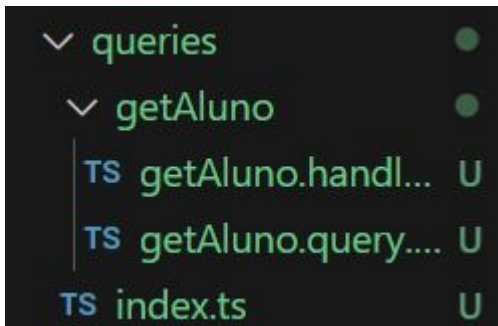
Escrita

- Insert(dado)
- Update(id, dado)
- Delete(id)



CQRS

- npm install --save @nestjs/cqrs



```
@QueryHandler(GetAlunoQuery)
export class GetAlunoHandler
  implements IQueryHandler<GetAlunoQuery, Array<Aluno> | null>
{
  async execute(query: GetAlunoQuery): Promise<Array<Aluno>> {
    return Aluno.findAll({
      attributes: {
        exclude: ['created_at']
      },
      where: [{
        id: query.id
      }]
    });
  }
}
```



CQRS

- `npm install --save @nestjs/cqrs`

```
export class GetAlunoQuery {  
  id: number;  
}
```

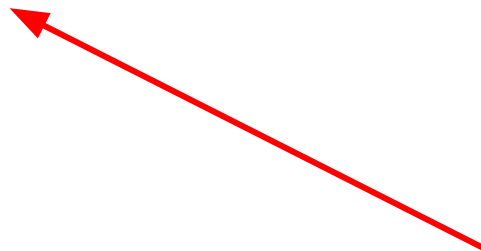
```
▼ queries  
  ▼ getAluno  
    TS getAluno.handl... U  
    TS getAluno.query... U  
    TS index.ts U
```

```
import { GetAlunoHandler } from "../getAluno/getAluno.handler";  
  
export const QueryHandlers = [GetAlunoHandler];
```



CQRS

```
@Get('/:id')
async findOne(@Param('id') id: string) {
  const query = plainToClass(GetAlunoQuery, { id: Number(id) });
  const aluno = await this.queryBus.execute(query);
  return aluno;
}
```



```
@Module({
  imports: [
    BullModule.registerQueue({ name: 'alunos' }),
    CqrsModule
  ],
  controllers: [AlunoController],
  providers: [AlunoScheduler, ...QueryHandlers],
})
export class AlunoModule {}
```



CQRS

```
export class CreateAlunoCommand {  
  nome: string;  
  matricula: string;  
}
```

```
@CommandHandler(CreateAlunoCommand)  
export class CreateAlunoHandler  
  implements ICommandHandler<CreateAlunoCommand, Aluno>{  
  
  async execute(aluno: CreateAlunoCommand): Promise<Aluno> {  
    let novo_aluno = await Aluno.create({  
      nome: aluno.nome,  
      matricula: aluno.matricula  
    });  
  
    return novo_aluno[0];  
  }  
}
```

```
✓ commands  
  ✓ criarAluno  
    TS createAluno.command.ts  
    TS createAluno.handler.ts  
  TS index.ts
```

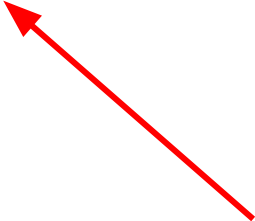
```
import { CreateAlunoHandler } from "../criarAluno/createAluno.handler";  
  
export const CommandHandlers = [  
  CreateAlunoHandler  
];
```



CQRS

```
@Post()  
async create(@Body() createAlunoDto: CreateAlunoDto) {  
  const command = plainToClass(CreateAlunoCommand, createAlunoDto);  
  const id = await this.commandBus.execute(command);  
  return id;  
}
```

```
@Module({  
  imports: [  
    BullModule.registerQueue({ name: 'alunos' }),  
    CqrsModule  
  ],  
  controllers: [AlunoController],  
  providers: [AlunoScheduler, ...QueryHandlers, ...CommandHandlers],  
})  
export class AlunoModule {}
```





Eventos

- Implementação simples de observador
- Permite que eventos sejam assinados e escutados
- `npm i --save @nestjs/event-emitter`

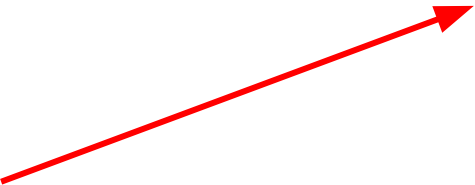
```
@Module({  
  imports: [  
    AlunoModule,  
    EventEmitterModule.forRoot(),  
    ScheduleModule.forRoot(),  
    BullModule.forRoot({
```




Eventos

```
@Injectable()
export class AlunoService {
  constructor(
    private event: EventEmitterService,
    private readonly queryBus: QueryBus,
    private readonly commandBus: CommandBus,
  ) { }
  async createUser(createAlunoDto: CreateAlunoDto): Promise<Aluno> {
    const command = plainToClass(CreateAlunoCommand, createAlunoDto);
    const id = await this.commandBus.execute(command);

    this.event.sendEmail(createAlunoDto);
    return id;
  }
}
```



```
import { Injectable } from '@nestjs/common';
import { EventEmitter2, OnEvent } from '@nestjs/event-emitter';
import { CreateAlunoDto } from '../dto/listar-aluno.dto';

@Injectable()
export class EventEmitterService {
  constructor(private eventEmitter: EventEmitter2) { }

  sendEmail(createAlunoDto: CreateAlunoDto): void {
    this.eventEmitter.emit('user.criado', createAlunoDto);
  }

  @OnEvent('user.criado')
  emailNewUser(createAlunoDto: CreateAlunoDto): void {
    console.log(`Envia email para ${createAlunoDto.nome} confir`);
  }
}
```



POST localhost:3000/aluno Send

Status: 201 Created Size: 0 Bytes Time: 49 ms

Query Headers² Auth Body¹ Tests Pre Run

Response Headers⁴ Cookies Results Docs

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "nome": "Diego",
3   "matricula": "123"
4 }
```

1

```
26 @Post()
27 async create(@Body() createAlunoDto: CreateAlunoDto) {
28   return this.alunoService.createUser(createAlunoDto);
29 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

node - app-geral

_at" TIMESTAMP WITH TIME ZONE NOT NULL, "updated_at" TIMESTAMP WITH TIME ZONE NOT NULL, PRIMARY KEY ("id"));

Executing (default): SELECT i.relname AS name, ix.indisprimary AS primary, ix.indisunique AS unique, ix.indkey AS indkey, array_ag

g(a.attnum) as column_indexes, array_agg(a.attname) AS column_names, pg_get_indexdef(ix.indexrelid) AS definition FROM pg_class t,

pg_class i, pg_index ix, pg_attribute a WHERE t.oid = ix.indrelid AND i.oid = ix.indexrelid AND a.attrelid = t.oid AND t.relkind

= 'r' and t.relname = 'aluno' GROUP BY i.relname, ix.indexrelid, ix.indisprimary, ix.indisunique, ix.indkey ORDER BY i.relname;

Database & tables created!

Executing (default): INSERT INTO "aluno" ("id","nome","matricula","created_at","updated_at") VALUES (DEFAULT,\$1,\$2,\$3,\$4) RETURNIN

G "id","nome","matricula","created_at","updated_at";

Envia email para Diego confirmando o cadastro



Queues

- Desafios de escala e desempenho
 - Suavizar picos de processamento
 - Backlog de requisições / tarefas
 - Adicionar e extrair tarefas na fila
 - Consumidor e produtor
 - Ciclo de vida
 - Retry
 - Tratamento de tarefas
 - FIFO, LIFO



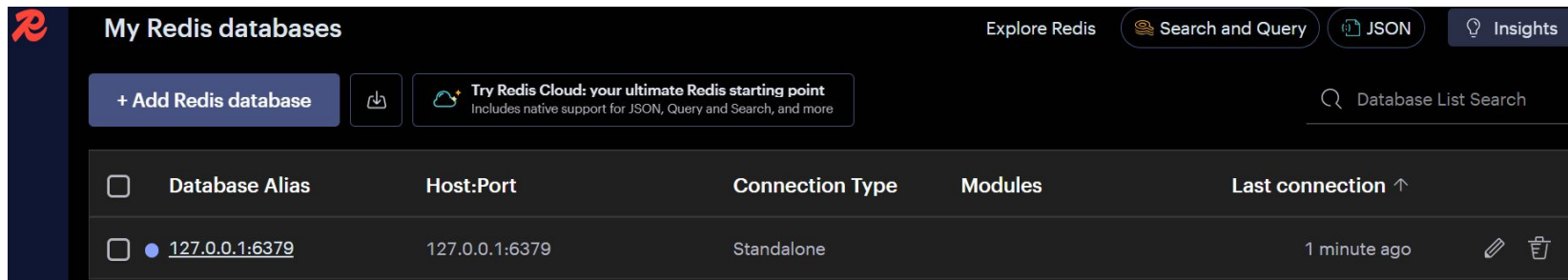
Queues

- `npm install --save @nestjs/bull bull`
 - Utiliza o Redis para persistir dados
 - Arquitetura distribuída e independente de plataforma
 - Consumidores e produtores diferentes



Queues

- docker pull redis
- docker run --name redis-api -p 6379:6379 -d redis
- RedisInsight

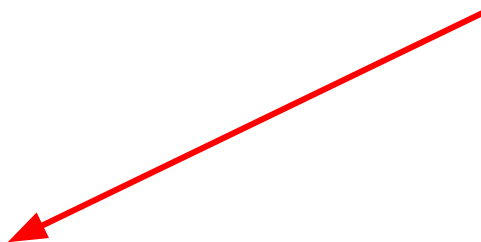




Queues

```
@Module({
  imports: [
    AlunoModule,
    ScheduleModule.forRoot(),
    BullModule.forRoot({
      redis: {
        host: 'localhost',
        port: 6379,
      },
      defaultJobOptions: {
        removeOnComplete: 100, // remover depois de terminar
        removeOnFail: 100, // remover depois de falhar
        attempts: 3, // retry
        backoff: {
          type: 'exponential',
          delay: 1000
        }
      }
    })
  ],
})
```

```
@Module({
  imports: [
    BullModule.registerQueue({ name: 'alunos' })
  ],
  controllers: [AlunoController],
  providers: [AlunoService, AlunoScheduler],
})
export class AlunoModule {}
```





Queues

```
export class SendEmailEvent {  
  constructor(  
    public id: number  
  ) {}  
}
```

▼ events ●
▼ email ●
TS send-email.event.ts U
TS send-email.handler.ts U
TS envia-email.ts U
TS index.ts U

```
@EventHandler(SendEmailEvent)  
@Processor('fila-alunos')  
export class SendEmailHandler  
  implements IEventHandler<SendEmailEvent>  
{  
  constructor(  
    @InjectQueue('fila-alunos')  
    private readonly fila: Queue,  
    private readonly alunoService: AlunoService  
  ) {}  
  
  async handle(event: SendEmailEvent) {  
    await this.fila.add('send-email-cadastro', event)  
  }  
  
  @Process('send-email-cadastro')  
  async process(job: Job<SendEmailEvent>){  
    let event = job.data;  
    let aluno = await this.alunoService.findOne(event.id);  
    console.log(`Enviando email para ${aluno[0].nome}`);  
  }  
}
```




Queues

```
constructor(  
  private event: EventEmitterService,  
  private readonly queryBus: QueryBus,  
  private readonly commandBus: CommandBus,  
  private readonly eventBus: EventBus  
) {  
  
  async createUser(createAlunoDto: CreateAlunoDto): Promise<Aluno> {  
  
    const command = plainToClass(CreateAlunoCommand, createAlunoDto);  
    const aluno = await this.commandBus.execute(command);  
  
    //this.event.sendEmail(createAlunoDto);  
    await this.eventBus.publish(new SendEmailEvent(aluno.id))  
    return aluno.id;  
  }  
}
```

- ▼ events
- ▼ email
 - TS send-email.event.ts U
 - TS send-email.handler.ts U
 - TS envia-email.ts U
 - TS index.ts U

```
@Module({  
  imports: [  
    BullModule.registerQueue({ name: 'fila-alunos' }),  
    CqrsModule  
  ],  
  controllers: [AlunoController],  
  providers: [  
    EventEmitterService,  
    AlunoScheduler,  
    AlunoService,  
    ...QueryHandlers,  
    ...CommandHandlers,  
    ...EventHandlers  
  ]  
})  
export class AlunoModule {}
```

```
import { SendEmailHandler } from "../email/send-email.handler";  
export const EventHandlers = [SendEmailHandler];
```




Caching

- npm install @nestjs/cache-manager cache-manager

```
@Module({  
  imports: [  
    AlunoModule,  
    CacheModule.register(),  
    EventEmitterModule.forRoot(),  
    ScheduleModule.forRoot(),  
    BullModule.forRoot({
```

```
CacheModule.register({ isGlobal: true })),
```

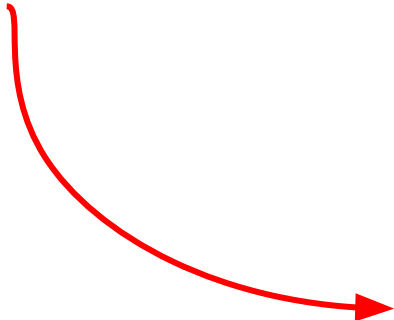
- Chave automática
- @CacheKey('custom-key')

```
import { CacheInterceptor, CacheTTL } from '@nestjs/cache-manager';  
import { Controller, Get, Post, Body, Param, UseInterceptors } from '@nestjs/common';  
  
@ApiTags('Alunos')  
@Controller('aluno')  
export class AlunoController {  
  constructor(  
    private readonly queryBus: QueryBus,  
    private readonly commandBus: CommandBus,  
    private readonly alunoService: AlunoService  
  ) {}  
  
  @UseInterceptors(CacheInterceptor)  
  @CacheTTL(30)  
  @Get(':id')  
  async findOne(@Param('id') id: string) {  
    return this.alunoService.findOne(+id);  
  }  
}
```



Caching

```
@Inject(CACHE_MANAGER) private cacheManager: Cache,
```



```
async findOne(id: number): Promise<Aluno> {  
  const cachedAluno: Aluno = await this.cacheManager.get(id.toString());  
  if (cachedAluno) {  
    console.log('Aluno retornado do cache:', cachedAluno);  
    return cachedAluno;  
  }  
  
  const query = plainToClass(GetAlunoQuery, { id: Number(id) });  
  const aluno = await this.queryBus.execute(query);  
  
  await this.cacheManager.set(id.toString(), aluno);  
  console.log('Aluno armazenado no cache:', aluno);  
  
  return aluno;  
}
```

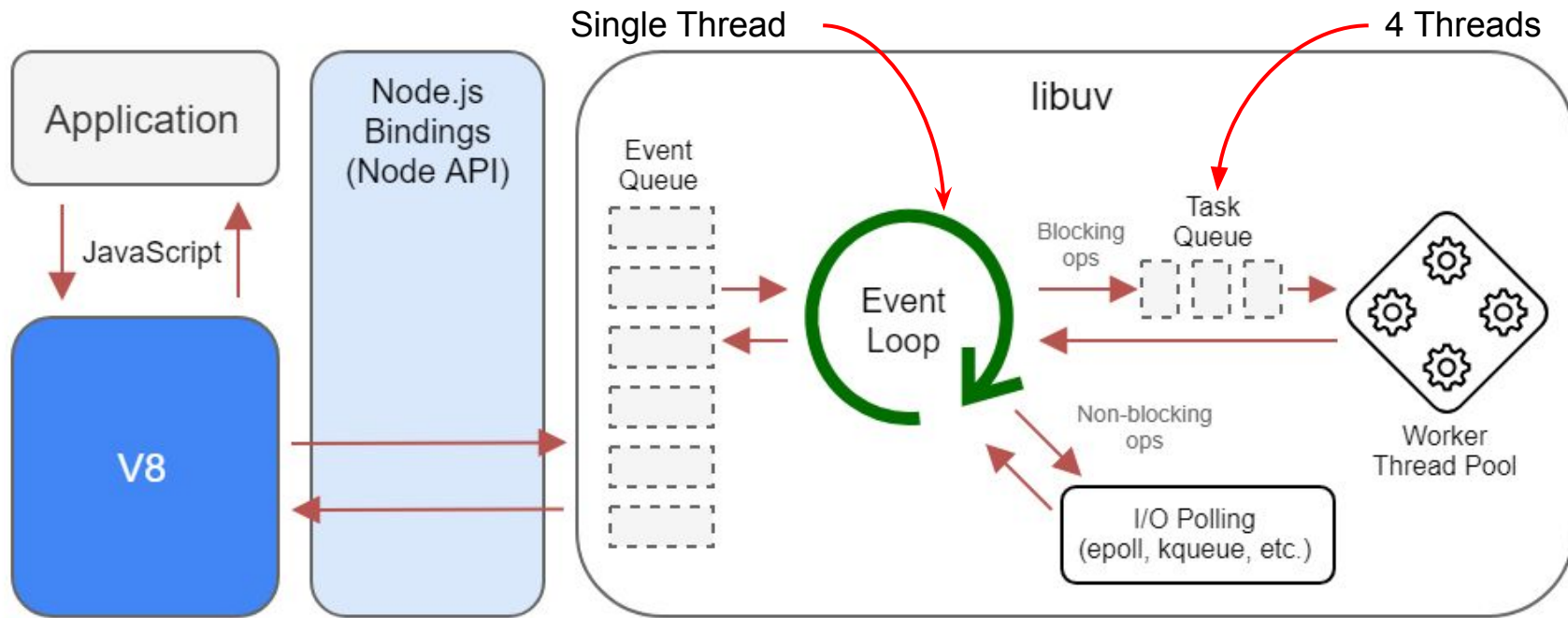


Threads

- Node executa código em um único thread
- Isso significa que o aplicativo só pode realizar uma tarefa por vez
- Internamente, o Node.js possui vários threads
 - Threads ocultas por meio da biblioteca libuv
 - Métodos assíncronos sem bloquear a thread principal



Threads





Threads

- Com CPUs disponíveis
 - Benefício de executar vários processos para melhorar o desempenho por meio do compartilhamento de carga
- O Node possui um módulo de cluster que torna o trabalho de compartilhamento de carga muito fácil



Threads

- Esse módulo cluster permite a fácil criação de processos filhos que compartilham portas de servidor
- O gerenciamento da conexão TCP e o compartilhamento de carga são feitos pelo módulo cluster



Threads



```
1
2 async function bootstrap() {
3   const app = await NestFactory.create(AppModule);
4
5   const config = new DocumentBuilder()
6     .setTitle('Cats example')
7     .setDescription('The cats API description')
8     .setVersion('1.0')
9     .addTag('cats')
10    .build();
11
12   const document = SwaggerModule.createDocument(app, config);
13   SwaggerModule.setup('api', app, document);
14
15   await app.listen(3000);
16 }
17
18 ClusterService.clusterize(bootstrap);
```



```
1 import { Injectable } from '@nestjs/common';
2
3 const cluster = require('cluster');
4 import * as process from 'node:process';
5
6 const numCPUs = parseInt(process.argv[2] || "1");
7
8 @Injectable()
9 export class ClusterService {
10   static clusterize(callback: Function): void {
11     if (cluster.isMaster) {
12       console.log(`MASTER SERVER (${process.pid}) IS RUNNING`);
13
14       for (let i = 0; i < numCPUs; i++) {
15         cluster.fork();
16       }
17
18       cluster.on('exit', (worker, code, signal) => {
19         console.log(`worker ${worker.process.pid} died`);
20       });
21     } else {
22       callback();
23     }
24   }
25 }
```




Threads

- Uma e duas threads

```
6 const numCPUs = parseInt(process.argv[2] || "1");
7
8 @Injectable()

[15:24:28] File change detected. Starting incremental compilation...

[15:24:28] Found 0 errors. Watching for file changes.

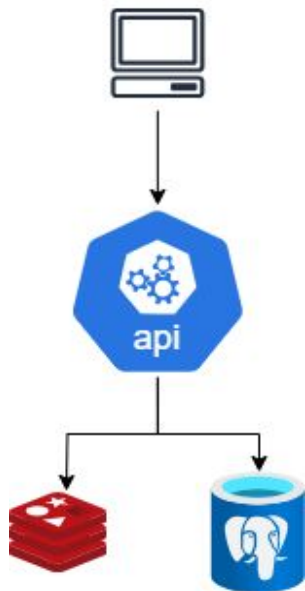
MASTER SERVER (10292) IS RUNNING
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [NestFactory] Starting Nest application...
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [InstanceLoader] AppModule dependencies initialized +24ms
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [InstanceLoader] AlunoModule dependencies initialized +0ms
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [RoutesResolver] ApplicationController {/}: +47ms
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [RouterExplorer] Mapped {/, GET} route +9ms
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [RoutesResolver] AlunoController {/aluno}: +1ms
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [RouterExplorer] Mapped {/aluno, POST} route +1ms
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [RouterExplorer] Mapped {/aluno, GET} route +1ms
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [RouterExplorer] Mapped {/aluno/:id, GET} route +1ms
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [RouterExplorer] Mapped {/aluno/:id, PATCH} route +1ms
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [RouterExplorer] Mapped {/aluno/:id, DELETE} route +0ms
[Nest] 15124 - 16/05/2024, 15:24:33 LOG [NestApplication] Nest application successfully started +7ms
```

```
5
6 const numCPUs = parseInt(process.argv[2] || "2");
7
8 @Injectable()

[15:23:41] Found 0 errors. Watching for file changes.

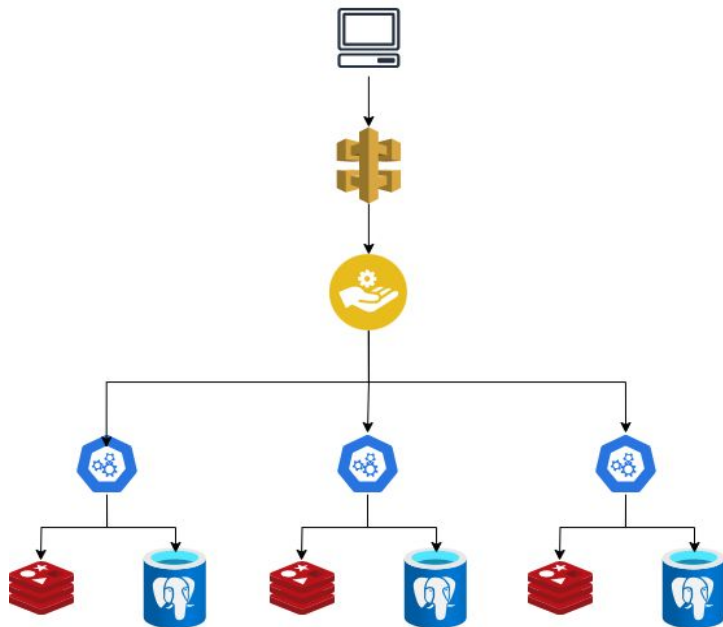
MASTER SERVER (19796) IS RUNNING
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [NestFactory] Starting Nest application...
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [InstanceLoader] AppModule dependencies initialized +31ms
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [InstanceLoader] AlunoModule dependencies initialized +1ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [NestFactory] Starting Nest application...
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [InstanceLoader] AppModule dependencies initialized +34ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [InstanceLoader] AlunoModule dependencies initialized +0ms
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [RoutesResolver] ApplicationController {/}: +57ms
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/, GET} route +7ms
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [RoutesResolver] AlunoController {/aluno}: +0ms
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/aluno, POST} route +2ms
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/aluno, GET} route +1ms
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/aluno/:id, GET} route +1ms
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/aluno/:id, PATCH} route +3ms
[Nest] 13044 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/aluno/:id, DELETE} route +1ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [NestApplication] Nest application successfully started +4ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [RoutesResolver] ApplicationController {/}: +53ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/, GET} route +4ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [RoutesResolver] AlunoController {/aluno}: +1ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/aluno, POST} route +2ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/aluno, GET} route +1ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/aluno/:id, GET} route +2ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/aluno/:id, PATCH} route +3ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [RouterExplorer] Mapped {/aluno/:id, DELETE} route +1ms
[Nest] 14960 - 16/05/2024, 15:23:46 LOG [NestApplication] Nest application successfully started +6ms
```


Monolito



- Único aplicativo com todos nossos serviços
- Facilidade para utilizar serviços
- Escalar a aplicação como um todo
- Caso um venha a falhar
 - **Todos irão falhar**

Microserviços



- Maior complexidade
 - Mais planejamento
 - Comunicação
 - Rastrear trocas de dados
- Funcionalidade separadas



Passo a passo

- Instalar o cliente do nest
 - `npm install -g @nestjs/cli`
- Criar um novo projeto nest
 - `nest new meu-projeto`
- Rodar o projeto
 - `npm run start:dev`



Passo a passo

- nest g resource users
 - Vai criar
 - Module
 - Controller
 - Service
 - DTO



Passo a passo

- nest g resource users
 - Vai criar
 - Module
 - Controller
 - Service
 - DTO →

npm install class-validator
class-transformer

```
import { IsString, IsEmail } from 'class-validator';

export class CreateUserDto {
  @IsString()
  name: string;

  @IsEmail()
  email: string;
}
```



Passo a passo

- nest g resource users

- Vai criar

- Module
- Controller
- Service →
- DTO

```
import { Injectable } from '@nestjs/common';
import { CreateUserDto } from '../dto/create-user.dto';
import { v4 as uuid } from 'uuid';

@Injectable()
export class UsersService {
  private users = [];

  create(dto: CreateUserDto) {
    const user = { id: uuid(), ...dto };
    this.users.push(user);
    return user;
  }

  findAll() {
    return this.users;
  }

  findOne(id: string) {
    return this.users.find(u => u.id === id);
  }
}
```



Passo a passo

- nest g resource users

- Vai criar

- Module
- Controller →
- Service
- DTO

```
import { Controller, Post, Body, Get, Param } from '@nestjs/common';
import { UsersService } from '../users.service';
import { CreateUserDto } from '../dto/create-user.dto';

@Controller('users')
export class UsersController {
  constructor(private readonly svc: UsersService) {}

  @Post()
  create(@Body() dto: CreateUserDto) {
    return this.svc.create(dto);
  }

  @Get()
  findAll() {
    return this.svc.findAll();
  }

  @Get(':id')
  findOne(@Param('id') id: string) {
    return this.svc.findOne(id);
  }
}
```



Passo a passo

- Main
 - Validação automática
 - `import { ValidationPipe } from '@nestjs/common';`
 - `app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));`



Passo a passo

- Adicionando o Swagger
 - npm install @nestjs/swagger

```
import { SwaggerModule, DocumentBuilder } from '@nestjs/swagger';

const config = new DocumentBuilder()
  .setTitle('Minha API')
  .setDescription('API exemplo com NestJS')
  .setVersion('1.0')
  .build();

const document = SwaggerModule.createDocument(app, config);
SwaggerModule.setup('api', app, document);
```



Passo a passo

- Modifique o DTO para incluir as opções do swagger

```
import { IsString, IsEmail } from 'class-validator';
import { ApiProperty } from '@nestjs/swagger';

export class CreateUserDto {
  @ApiProperty({
    description: 'Nome do usuário',
    example: 'Diego',
  })
  @IsString()
  name: string;

  @ApiProperty({
    description: 'Endereço de e-mail',
    example: 'diego.castro@cefet-rj.br',
  })
  @IsEmail()
  email: string;
}
```



Passo a passo

- npm install
@nestjs/sequelize
sequelize
sequelize-typescript
sqlite3

```
import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';
import { UsersModule } from './users/users.module';
import { User } from './users/user.model';

@Module({
  imports: [
    SequelizeModule.forRoot({
      dialect: 'sqlite',
      storage: 'database.sqlite',
      autoLoadModels: true,
      synchronize: true,
      models: [User],
    }),
    UsersModule,
  ],
})
export class AppModule {}
```



Passo a passo

- Crie a model de usuário para interagir com o banco de dados

```
import { Table, Column, Model, DataType } from 'sequelize-typescript';

@Table
export class User extends Model<User> {
  @Column({
    type: DataType.UUID,
    defaultValue: DataType.UUIDV4,
    primaryKey: true,
  })
  id: string;

  @Column({
    type: DataType.STRING,
    allowNull: false,
  })
  name: string;

  @Column({
    type: DataType.STRING,
    allowNull: false,
    unique: true,
  })
  email: string;
}
```



Passo a passo

- Atualize o module da entidade para incluir a opção de uso do sequelize como orm

```
import { Module } from '@nestjs/common';
import { SequelizeModule } from '@nestjs/sequelize';
import { UsersService } from './users.service';
import { UsersController } from './users.controller';
import { User } from './user.model';

@Module({
  imports: [SequelizeModule.forRoot(),
    SequelizeModule.forFeature([User])],
  controllers: [UsersController],
  providers: [UsersService],
})
export class UsersModule {}
```



Passo a passo

- Crie a camada de repositório para acesso ao banco de dados

```
import { Injectable } from '@nestjs/common';
import { InjectModel } from '@nestjs/sequelize';
import { User } from '../user.model';
import { CreateUserDto } from '../dto/create-user.dto';

@Injectable()
export class UsersRepository {
  constructor(
    @InjectModel(User)
    private readonly userModel: typeof User,
  ) {}

  async create(dto: CreateUserDto): Promise<User> {
    return this.userModel.create(dto);
  }

  async findAll(): Promise<User[]> {
    return this.userModel.findAll();
  }

  async findOne(id: string): Promise<User | null> {
    return this.userModel.findByPk(id);
  }
}
```



Passo a passo

- Edite a camada de service para fazer a chamada do repositório

```
import { Injectable } from '@nestjs/common';
import { UsersRepository } from '../users.repository';
import { CreateUserDto } from '../dto/create-user.dto';

@Injectable()
export class UsersService {
  constructor(private readonly repo: UsersRepository) {}

  create(dto: CreateUserDto) {
    return this.repo.create(dto);
  }

  findAll() {
    return this.repo.findAll();
  }

  findOne(id: string) {
    return this.repo.findOne(id);
  }
}
```